

Glosarium Istilah Membuat LLM Sendiri

Pendamping praktis untuk proyek mini-LLM Bahasa Indonesia — RTX 4050 6GB, 32 juta parameter

Cara pakai. Jangan dibaca sekali habis, dan jangan dihafal. Baca sekali sambil lalu supaya kenal bentuknya, lalu buka lagi setiap ketemu istilah yang bikin mentok. Otak memang terasa penuh di awal — itu normal, dan bukan tanda Anda tidak paham. Istilah teknis sengaja tidak diterjemahkan, karena itu nama konsep, bukan kata Inggris biasa.

Baris bertanda ★ adalah istilah inti. Kalau waktu terbatas, kuasai yang bertanda bintang saja dulu — sisanya akan menyusul sendiri sambil jalan.

Baris → **hijau** menghubungkan istilah itu dengan angka konkret di proyek Anda, supaya tidak berhenti jadi teori.

Glosarium ini melengkapi *glosarium-istilah-llm.pdf* yang sudah Anda punya. Yang itu untuk *membaca* tech report model raksasa; yang ini untuk *membangun* model sendiri dari nol.

KARTU CEPAT — ANGKA PROYEK ANDA	
Parameter model utama	32.0 juta
Lebar model (d_model)	512
Kedalaman (n_layer)	8 blok
Attention head	8 query / 4 key-value (GQA)
Ukuran vocab	16.384 token
Panjang konteks	512 token
Target token training	~1,3 miliar
VRAM GPU total / bebas	6,0 GB / ~4,6 GB
Perkiraan VRAM dipakai	~2,0 GB (batch 16)
Loss saat awal (harus)	9,70 = $\ln(16.384)$
Perkiraan lama training	8-11 jam

1. Fondasi — apa yang sebenarnya dikerjakan LLM

Kalau hanya sempat paham satu bagian, pahami yang ini.

★ Token token	Potongan teks terkecil yang diproses model. Bukan huruf, bukan juga kata utuh. Kata “mengerjakan” bisa pecah jadi <i>men</i> g + <i>er</i> ja + <i>kan</i> . Semua yang dilakukan model — membaca dan menulis — terjadi dalam satuan token. → <i>Model Anda mengenal 16.384 token berbeda.</i>
★ Parameter parameter / weight	Angka di dalam model yang nilainya berubah selama training. Awalnya acak, lambat laun jadi “pengetahuan” model. Jumlah parameter = ukuran kapasitas otak model. → <i>Model Anda punya 32 juta. GPT-3 punya 175 miliar — sekitar 5.500x lebih banyak.</i>
★ Next-token prediction prediksi token berikutnya	Satu-satunya hal yang dilatih pada LLM: menebak token berikutnya dari token-token sebelumnya. Terdengar sepele, tapi seluruh kemampuan model — tata bahasa, fakta, gaya bicara — muncul sebagai efek samping dari latihan menebak ini.
★ Autoregressive autoregressive	Cara model menghasilkan teks: keluarkan satu token, tempelkan ke input, lalu pakai untuk menebak token berikutnya. Diulang terus. Karena itu LLM menulis sepotong demi sepotong, bukan sekaligus.
Embedding embedding	Tabel besar yang mengubah ID token (misalnya angka 4127) menjadi deretan angka yang bermakna bagi model. Ibarat kamus: dari nomor entri jadi arti. → <i>Ukuran tabel Anda: $16.384 \times 512 = 8,4$ juta angka (26% dari seluruh model).</i>
Vektor & dimensi vector / dimension	Vektor = deretan angka. Dimensi = panjang deretan itu. Model bekerja dengan mengolah vektor, bukan huruf. → <i>$d_{\text{model}}=512$ artinya tiap token diwakili 512 angka.</i>
Logits logits	Skor mentah yang dikeluarkan model untuk setiap token di vocab — seberapa cocok tiap kandidat jadi token berikutnya. Belum berupa persentase. → <i>Tiap langkah, model Anda mengeluarkan 16.384 skor sekaligus.</i>
Softmax softmax	Rumus yang mengubah logits jadi probabilitas yang jumlahnya pas 100%. Ini jembatan dari “skor” ke “peluang”.
Training vs Inference training / inference	Training = proses menyetel parameter (berat, lama, butuh GPU besar). Inference = memakai model yang sudah jadi untuk menjawab (jauh lebih ringan). → <i>Training Anda ~2,0 GB VRAM; inference nanti hanya puluhan MB.</i>

2. Data & Tokenizer

Kualitas data lebih menentukan hasil daripada ukuran model.

Korpus corpus	Kumpulan teks mentah yang dipakai untuk melatih model. → <i>Rencana Anda: Wikipedia Indonesia + teks web + berita, ~400 juta token unik.</i>
★ Tokenizer tokenizer	Penerjemah dua arah antara teks dan angka. Model tidak pernah melihat huruf — hanya ID token dari tokenizer.
★ BPE Byte Pair Encoding	Algoritma pembuat tokenizer. Mulai dari byte satuan, lalu berulang kali menggabungkan pasangan yang paling sering muncul bersebelahan. Hasilnya: kata umum jadi satu token, kata langka pecah jadi beberapa.
★ Vocab size ukuran kosakata	Jumlah token unik yang dikenal tokenizer. Ini keputusan besar: vocab besar membuat teks lebih ringkas, tapi memakan banyak parameter untuk tabel embedding. → <i>Anda pilih 16.384. Kalau pakai 50.257 (GPT-2), 56% model habis hanya untuk tabel.</i>
Rasio kompresi compression ratio	Rata-rata berapa karakter yang muat dalam satu token. Makin tinggi makin efisien — artinya model melihat lebih banyak isi dengan anggaran token yang sama. → <i>Target 3,5-4 karakter/token. Tokenizer Inggris hanya dapat ~2,2 untuk teks Indonesia.</i>
Special token special token	Token buatan yang tidak mewakili teks biasa, melainkan penanda struktur: awal dokumen, akhir dokumen, giliran bicara. → <i>Anda pakai 8 aktif (< bos >, < user >, < assistant >, dst) + 24 slot cadangan.</i>

★ Deduplikasi deduplication / dedup	Membuang dokumen yang isinya sama atau nyaris sama. Langkah dengan hasil terbesar per usaha di seluruh pipeline data: data kembar membuat model menghafal, bukan belajar — dan membuat nilai ujiannya berbohong.
MinHash / LSH MinHash	Teknik untuk mencari dokumen yang <i>mirip</i> (bukan cuma persis sama) di antara jutaan dokumen, tanpa harus membandingkan semuanya satu per satu.
★ Train / validation split train / val split	Memisahkan data jadi bagian latihan dan bagian ujian. Model tidak boleh pernah melihat data ujian saat belajar — kalau tidak, nilainya tidak berarti apa-apa.
Data leakage kebocoran data	Data ujian ikut masuk ke data latihan. Akibatnya loss terlihat bagus padahal model cuma menghafal. Salah satu kesalahan paling menipu.
★ Epoch vs Step epoch / step	Epoch = satu putaran penuh melewati seluruh dataset. Step = satu kali parameter diperbarui. Satu epoch berisi ribuan step. → <i>Rencana Anda: 3 epoch di atas ~400 juta token unik.</i>
memmap memory-mapped file	Cara membaca file raksasa langsung dari disk, sepotong demi sepotong, tanpa memuat seluruhnya ke RAM. → <i>Dipakai agar dataset 800 MB tidak membebani RAM 20 GB Anda.</i>
uint16 unsigned 16-bit integer	Tipe angka berukuran 2 byte yang bisa menyimpan nilai 0–65.535. Cukup untuk menyimpan ID token, dan separuh lebih hemat dibanding 4 byte. → <i>Bisa dipakai karena vocab Anda (16.384) di bawah 65.536.</i>

3. Arsitektur — isi dalam model

Bagian yang akan Anda tulis sendiri baris demi baris di Fase 3.

★ Transformer Transformer	Arsitektur dasar hampir semua LLM modern. Intinya menumpuk blok yang isinya selalu dua hal: attention (mencampur informasi antar token) dan FFN (mengolah tiap token sendiri-sendiri).
Layer / blok layer / block	Satu unit pemrosesan lengkap. Ditumpuk berkali-kali; makin dalam, makin banyak tahap penalaran yang bisa dilakukan. → <i>Model Anda: 8 blok bertumpuk.</i>
d_model model dimension / hidden size	Lebar model — berapa angka dipakai untuk mewakili satu token di sepanjang jalur utama. → <i>Anda: 512.</i>
★ Residual stream residual stream	Jalur utama tempat informasi mengalir dari input ke output. Tiap blok <i>menambahkan</i> hasil olahannya ke jalur ini, tidak menyimpannya. Bayangkan papan tulis yang terus dicoret-tambahi tiap lapisan, bukan dihapus total.
Residual connection skip connection	Penjumlahan yang membentuk residual stream tadi. Inilah yang membuat model sangat dalam tetap bisa dilatih — gradient punya jalan pintas untuk mengalir sampai ke lapisan bawah.
★ Attention attention	Mekanisme yang membuat tiap token bisa “menoleh” ke token lain dan mengambil konteks darinya. Ini yang membuat model paham bahwa “-nya” di akhir kalimat merujuk ke subjek di awal kalimat.
★ Query, Key, Value Q / K / V	Tiga peran yang dihitung dari tiap token. Analogi perpustakaan: Query = “saya sedang cari apa”, Key = “label yang saya pasang”, Value = “isi yang saya berikan kalau dipilih”. Q dicocokkan ke semua K, lalu V yang cocok diambil.
Attention head head	Satu “sudut pandang” attention. Beberapa head jalan paralel: satu bisa fokus ke kata sebelumnya, satu ke subjek kalimat, dan seterusnya. → <i>Anda: 8 head, masing-masing bekerja di 64 dimensi (512 dibagi 8).</i>
MHA / GQA / MQA Multi-Head / Grouped-Query / Multi-Query Attention	Tiga varian. MHA : tiap head punya K dan V sendiri (boros memori saat inference). MQA : semua head berbagi satu K/V (hemat, tapi kualitas turun). GQA : jalan tengah — beberapa head berbagi satu set K/V. → <i>Anda pakai GQA: 8 head query berbagi 4 set K/V, memangkas KV-cache separuh.</i>
★ Causal mask causal / attention mask	Larangan bagi token untuk melihat token di masa depan. Tanpa ini, model bisa “menyontek” jawaban dan loss-nya terlihat sangat bagus padahal modelnya tidak belajar apa-apa. → <i>Bug di sini bikin loss awal ~2-3 padahal seharusnya 9,70.</i>

Context length context window / seq_len	Berapa token yang bisa dibaca model sekaligus. Di luar itu, model tidak ingat apa-apa. → <i>Anda: 512 token saat pretrain, dinaikkan ke 1024 saat SFT.</i>
★ RoPE Rotary Position Embedding	Cara memberi tahu model urutan kata, dengan memutar vektor token sesuai posisinya. Tanpa informasi posisi, attention tidak bisa membedakan “anjing menggigit orang” dari “orang menggigit anjing”.
LayerNorm / RMSNorm normalization	Penjaga agar angka di dalam model tidak membesar atau mengecil tak terkendali. RMSNorm versi lebih sederhana dan cepat, dipakai hampir semua model baru.
Pre-norm vs Post-norm pre-norm / post-norm	Apakah normalisasi dipasang sebelum atau sesudah sub-layer. Pre-norm jauh lebih stabil untuk model dalam, dan jadi standar sekarang. → <i>Anda pakai pre-norm.</i>
★ FFN / MLP Feed-Forward Network	Lapisan yang mengolah tiap token secara terpisah, tanpa melihat token lain. Riset interpretabilitas menunjukkan di sinilah sebagian besar “pengetahuan” model tersimpan. → <i>Menghabiskan 54% parameter model Anda — porsi terbesar.</i>
SwiGLU / GELU / ReLU activation function	Fungsi non-linear di dalam FFN. Tanpa non-linearitas, menumpuk 8 layer sama saja dengan 1 layer. SwiGLU varian terbaik saat ini pada jumlah parameter setara.
Weight tying tied embeddings	Memakai satu tabel bobot yang sama untuk mengubah token jadi vektor (input) dan vektor jadi skor (output). Masuk akal karena keduanya memetakan hal yang sama. → <i>Menghemat 8,4 juta parameter — 26% dari model Anda.</i>
Inisialisasi initialization	Nilai acak awal parameter sebelum training. Terdengar sepele, tapi salah skala di sini bisa membuat training gagal total sejak langkah pertama.
Flash Attention / SDPA Flash Attention	Implementasi attention yang tidak pernah menyimpan matriks besar berukuran panjang-teks-kuadrat di memori. Inilah yang membuat GPU 6 GB sanggup.

4. Training — proses belajarnya

Semua yang terjadi berulang ribuan kali selama 8-11 jam nanti.

★ Loss loss	Angka yang mengukur seberapa salah tebakan model. Makin kecil makin baik. Seluruh training pada dasarnya adalah usaha menurunkan angka ini. → <i>Loss Anda harus mulai dari 9,70 lalu turun. Sekitar 3-4 sudah bagus untuk 32M.</i>
Cross-entropy cross-entropy loss	Rumus loss standar untuk menebak token. Intinya: “seberapa terkejut model melihat token yang ternyata benar?” Makin terkejut, makin besar loss-nya.
Perplexity perplexity	Loss yang diubah jadi bentuk lebih intuitif (e pangkat loss). Artinya: “model sebingung orang yang harus menebak di antara sekian pilihan.” Perplexity 20 = seolah menebak dari 20 kemungkinan.
★ Gradient gradient	Untuk tiap parameter: ke arah mana dan seberapa besar ia harus diubah agar loss turun. Analogi: kabut tebal di lereng gunung, gradient adalah kemiringan tanah di bawah kaki Anda.
Backpropagation backprop	Algoritma yang menghitung gradient untuk semua parameter sekaligus, dengan menelusuri mundur dari loss ke input.
Forward / backward pass forward / backward pass	Forward: data masuk, model menebak, loss dihitung. Backward: gradient dihitung mundur. Backward kira-kira 2x lebih mahal dari forward.
★ Optimizer / AdamW optimizer	Algoritma yang benar-benar memperbarui parameter berdasarkan gradient. AdamW standar de facto: ia mengingat rata-rata arah gradient dan seberapa besar goyangannya, lalu menyesuaikan langkah tiap parameter secara terpisah. → <i>Ingatan AdamW ini yang memakan 8 dari 16 byte per parameter.</i>
★ Learning rate LR	Seberapa besar langkah tiap kali parameter diperbarui. Terlalu besar: training meledak. Terlalu kecil: tidak selesai-selesai. Hyperparameter paling berpengaruh.
Warmup warmup	Menaikkan learning rate pelan-pelan di awal training, bukan langsung penuh. Di awal, parameter masih acak dan gradient sangat liar — langkah besar di sini bisa merusak model permanen. → <i>Rencana Anda: warmup di 2% langkah pertama.</i>
Cosine decay LR schedule	Menurunkan learning rate mengikuti kurva melandai sampai mendekati nol di akhir. Ibarat mengecilkan langkah saat sudah dekat tujuan agar tidak kelewatan.

Gradient clipping grad clipping	Membatasi besar gradient di ambang tertentu. Satu batch aneh bisa menghasilkan gradient raksasa yang merusak model — ini sabuk pengamanannya. → <i>Ambang standar: 1.0.</i>
Grad norm gradient norm	Ukuran besar gradient secara keseluruhan. Indikator kesehatan training: kalau melonjak liar atau jadi NaN, ada yang salah.
★ Batch size batch size	Berapa potong teks diproses bersamaan dalam satu langkah. Batch besar = gradient lebih halus dan GPU lebih efisien, tapi makan VRAM. → <i>Anda mulai dari 16; ini tombol pertama yang diputar kalau kena OOM.</i>
★ Gradient accumulation grad accumulation	Trik menumpuk gradient dari beberapa batch kecil sebelum memperbarui parameter. Hasil matematisnya <i>sama persis</i> dengan batch besar, tapi VRAM yang dipakai tetap sekecil batch kecil. Inilah cara GPU 6 GB meniru GPU besar.
Effective batch size batch efektif	$\text{batch_size} \times \text{gradient_accumulation}$. Ini angka yang sebenarnya menentukan kualitas gradient. → <i>Contoh: batch 16 x akumulasi 8 = batch efektif 128.</i>
★ Mixed precision / AMP Automatic Mixed Precision	Menghitung memakai angka presisi rendah (16-bit) supaya jauh lebih cepat, sambil menyimpan salinan presisi penuh agar tetap akurat.
fp32 / fp16 / bf16 format angka	fp32 4 byte, paling teliti. fp16 2 byte, cepat tapi mudah meluap/underflow. bf16 2 byte dengan jangkauan selebar fp32 — lebih aman dari fp16, tanpa perlu penanganan tambahan. → <i>RTX 4050 Anda mendukung bf16 secara native, jadi itu yang dipakai.</i>
Gradient checkpointing activation checkpointing	Membuang hasil antara saat forward, lalu menghitungnya ulang saat backward. Menukar kecepatan (~30% lebih lambat) demi memori (hemat 60-70% aktivasi). → <i>Katup pengaman kalau nanti VRAM mepet.</i>
Weight decay weight decay	Dorongan halus agar bobot tidak tumbuh membesar tanpa alasan. Bentuk regularisasi. → <i>Biasanya 0.1, tapi TIDAK diterapkan pada norm dan embedding.</i>
★ Overfitting overfitting	Model mulai menghafal data latihan alih-alih memahami polanya. Tandanya jelas: loss latihan terus turun sementara loss validasi mulai naik.
Checkpoint checkpoint	Simpanan lengkap kondisi training (bobot + optimizer + posisi data) agar bisa dilanjutkan kalau berhenti di tengah jalan. → <i>Wajib ada sebelum run semalam — laptop bisa sleep atau Windows restart.</i>

5. Memori GPU — kenapa 6 GB terasa sempit

Bagian paling menentukan seberapa besar model yang boleh Anda buat.

★ VRAM video RAM	Memori khusus di dalam GPU. Semua yang dipakai saat training harus muat di sini. RAM biasa tidak bisa menggantikannya. → <i>Anda punya 6,0 GB, tapi ~1,4 GB sudah dipakai desktop Windows. Sisa nyata ~4,6 GB.</i>
★ Optimizer state optimizer state	Isi VRAM yang jumlahnya tetap, tidak peduli batch: bobot (4 byte) + gradient (4 byte) + dua ingatan AdamW (8 byte) = 16 byte per parameter . Aturan praktisnya: 1 juta parameter = 16 MB. → <i>Model 32M Anda: ~490 MB. Model 350M butuh 5,6 GB — tidak muat sama sekali.</i>
★ Aktivasi activations	Hasil antara tiap layer yang harus disimpan sementara untuk dipakai saat backward. Besarnya tidak tergantung jumlah parameter, melainkan batch x panjang teks x lebar model x jumlah layer. → <i>Di batch 16: ~1,1 GB. Naik ke batch 32, jadi ~2,2 GB — kelipatan lurus.</i>
CUDA context CUDA context	Ongkos tetap yang diambil driver GPU begitu program dimulai, sebelum data apa pun dimuat. → <i>Sekitar 500 MB di Windows.</i>
OOM Out Of Memory	Error saat VRAM habis. Urutan penanganannya: kecilkan batch → naikan gradient accumulation → nyalakan gradient checkpointing → baru terakhir kecilkan model.
Fragmentasi memory fragmentation	VRAM total masih cukup, tapi terpecah jadi potongan kecil sehingga blok besar tidak dapat tempat. Mirip rak buku yang penuh celah tapi tak muat buku tebal.

6. Skala & Compute — berapa lama dan seberapa besar

Cara memperkirakan sebelum menghabiskan semalaman.

FLOPs floating point operations	Jumlah operasi hitung. Satuan biaya komputasi. Rumus praktis biaya training: 6 x jumlah parameter x jumlah token .
TFLOPS tera-FLOPS	Triliun operasi per detik — kecepatan GPU. Angka brosur vendor hampir selalu jauh di atas kenyataan.
★ MFU Model FLOPs Utilization	Persentase kemampuan GPU yang benar-benar terpakai. Model kecil susah mencapai MFU tinggi karena perkalian matriksnya terlalu kecil untuk mengisi GPU penuh. → <i>Harapan realistis di laptop Windows: 25-35%.</i>
Throughput token per detik	Kecepatan training sesungguhnya. Angka paling jujur untuk memperkirakan sisa waktu. → <i>Perkiraan Anda: 25.000-45.000 token/detik.</i>
★ Scaling law scaling law	Rumus empiris yang menghubungkan ukuran model, jumlah data, dan compute dengan kualitas hasil. Membuat kualitas model bisa diramalkan sebelum dilatih.
★ Chinchilla Chinchilla-optimal	Temuan bahwa titik paling efisien adalah sekitar 20 token data per parameter . Sebelum ini, orang membuat model besar tapi kekurangan data. → <i>Untuk 32M: 640 juta token. Anda menargetkan 1,3 miliar — sengaja 2x lipat.</i>
Over-training over-training	Sengaja melatih melebihi titik Chinchilla. Boros saat training, tapi menghasilkan model kecil yang lebih pintar — dan model kecil lebih murah dipakai seumur hidupnya. Praktik standar sekarang.
Under-trained under-trained	Model besar yang kurang data/waktu. Hasilnya bisa lebih buruk daripada model kecil yang dilatih tuntas. → <i>Alasan Anda memilih 32M matang, bukan 88M setengah jadi.</i>

7. Setelah pretrain — mengubahnya jadi lawan bicara

Pretrain menghasilkan model yang bisa berbahasa, bukan yang bisa menjawab.

★ Pretraining pretraining	Tahap pertama dan terlama: model belajar bahasa dari teks umum dalam jumlah besar. Hasilnya model yang lancar berbahasa tapi belum tahu cara menjawab pertanyaan — kalau diberi “halo”, ia mungkin malah melanjutkan menulis artikel.
Base model base / foundation model	Model hasil pretrain saja, belum diajari format percakapan.
Anneal / cooldown annealing / decay phase	Fase akhir pretrain: campuran data diganti ke yang berkualitas tinggi sambil learning rate diturunkan ke nol. Biayanya nyaris gratis tapi efeknya besar. → <i>Rencana Anda: 10% langkah terakhir.</i>
★ SFT Supervised Fine-Tuning	Tahap mengajari model format tanya-jawab, memakai kumpulan contoh percakapan. Di sinilah model berubah dari “penerus teks” jadi “asisten”. → <i>Rencana Anda: 20-50 ribu dialog, ~1 jam training.</i>
★ Chat template chat template	Format baku penanda giliran bicara, memakai special token. Tanpa ini model tidak punya cara tahu di mana ucapan user berakhir dan jawabannya harus mulai.
★ Loss masking loss masking	Menghitung loss hanya pada bagian jawaban asisten, bukan pada pertanyaan user. Kalau lupa, model malah belajar meniru cara user bertanya, bukan menjawabnya. Ini bug SFT paling sering terjadi.
Instruction tuning instruction tuning	Nama lain untuk SFT dengan penekanan pada kemampuan mengikuti perintah.
RLHF / DPO alignment	Tahap lanjutan: model diajari memilih jawaban yang lebih disukai manusia. Di luar cakupan proyek ini, tapi ini yang membuat model komersial terasa sopan dan membantu.
Catastrophic forgetting catastrophic forgetting	Model melupakan kemampuan lama saat dilatih hal baru. Alasan learning rate saat SFT dibuat jauh lebih kecil dari saat pretrain. → <i>SFT Anda memakai LR sekitar 10x lebih kecil.</i>

8. Inference — saat model dipakai

Bagian yang Anda tulis di Fase 7, dan yang akhirnya bisa diajak ngobrol.

★ KV-cache key-value cache	Menyimpan Key dan Value token-token sebelumnya supaya tidak dihitung ulang tiap kali satu token baru keluar. Tanpa ini, menghasilkan token ke-100 berarti memproses ulang 99 token sebelumnya — pemborosan besar. → Untuk satu percakapan 512 token, cuma butuh 4 MB berkat GQA.
Prefill vs Decode prefill / decode	Prefill : memproses seluruh prompt sekaligus (cepat, GPU terisi penuh). Decode : mengeluarkan token satu per satu (lambat, GPU banyak menganggur). Ini sebabnya jawaban panjang terasa lebih lama.
Greedy decoding greedy	Selalu ambil token dengan skor tertinggi. Hasilnya konsisten tapi cenderung membosankan dan gampang mengulang.
★ Temperature temperature	Pengatur keberanian model. Mendekati 0 = aman dan berulang. Sekitar 0,7-0,9 = wajar. Di atas 1,2 = liar dan sering ngawur.
Top-k top-k sampling	Hanya boleh memilih dari k token teratas. Membuang ekor kandidat yang aneh.
Top-p / nucleus nucleus sampling	Hanya memilih dari kandidat teratas yang probabilitasnya menumpuk sampai p (misal 90%). Lebih pintar dari top-k karena jumlah kandidatnya menyesuaikan situasi.
Repetition penalty repetition penalty	Menurunkan peluang token yang sudah muncul. Obat untuk model yang mengulang kalimat sama terus — penyakit khas model kecil.
Stop token EOS / stop token	Token penanda “berhenti di sini”. Tanpa penanganan yang benar, model akan terus menulis sampai batas panjang, termasuk mengarang giliran user.
★ Halusinasi hallucination	Model menyatakan sesuatu dengan yakin padahal salah. Bukan bug — konsekuensi langsung dari cara kerjanya yang menebak token paling mungkin, bukan mencari fakta. → Model 32M Anda akan sangat sering halusinasi. Itu wajar, bukan tanda kode Anda salah.
Streaming streaming	Menampilkan token begitu keluar, tidak menunggu kalimat selesai. Yang membuat chatbot terasa mengetik.
Kuantisasi quantization	Memampatkan bobot ke presisi lebih rendah (misal 8 atau 4 bit) agar model lebih kecil dan cepat, dengan sedikit penurunan kualitas.

9. Pasangan istilah yang sering tertukar

Kalau bingung, kemungkinan besar salah satu dari ini penyebabnya.

Parameter vs Token	Parameter = ukuran model (kapasitas otaknya). Token = jumlah data yang dibaca model. Dua sumbu berbeda: 32 juta parameter, 1,3 miliar token.
Epoch vs Step	Epoch = satu putaran seluruh dataset. Step = satu kali update parameter. Satu epoch berisi puluhan ribu step.
Batch size vs Effective batch	Batch size = yang muat di VRAM sekali jalan. Effective batch = batch size x akumulasi, dan inilah yang menentukan kualitas gradient.
Loss vs Perplexity	Angka yang sama dalam dua baju: perplexity = e pangkat loss. Loss untuk dilihat kurvanya, perplexity untuk dibicarakan ke orang.
Training vs Fine-tuning	Training (pretrain) = dari nol, mahal, lama. Fine-tuning (SFT) = melanjutkan model yang sudah jadi, murah, cepat.
d_model vs d_ffn	d_model = lebar jalur utama (512). d_ffn = lebar sementara di dalam FFN (1408), tempat model “berpikir lebih lebar” sebelum kembali menyempit.
Attention head vs Layer	Head = paralel, beberapa sudut pandang dalam satu waktu. Layer = seri, tahap demi tahap. Anda: 8 head (lebar) x 8 layer (dalam).
VRAM vs RAM	VRAM = memori di GPU (6 GB, ketat, penentu segalanya). RAM = memori sistem (20 GB, longgar). Keduanya tidak saling menggantikan.

Kalau bingung harus mulai dari mana

Dua belas istilah ini menjelaskan sekitar 80% dari sisanya.

1. Token 2. Parameter 3. Next-token prediction 4. Loss 5. Gradient 6. Learning rate 7. Batch size 8. Attention 9. Residual stream 10. VRAM / optimizer state 11. Pretrain vs SFT 12. KV-cache

Satu hal yang layak diingat saat merasa kewalahan: istilah-istilah ini terlihat banyak karena disajikan sekaligus, padahal dalam praktik Anda hanya akan berurusan dengan 5-10 istilah per fase. Saat menyiapkan data, Anda tidak perlu memikirkan KV-cache. Saat menulis attention, Anda tidak perlu memikirkan MFU. Plan proyek sengaja disusun bertahap justru supaya bebannya tidak datang bersamaan.

*Dan yang paling penting: **Anda tidak perlu paham semuanya sebelum mulai.** Pemahaman datang dari melihat angka bergerak di layar, bukan dari menghafal daftar istilah.*